

UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA FILHO”

Instituto de Ciência e Tecnologia - Câmpus de Sorocaba

Engenharia de Controle e Automação

SISTEMA DE TELEMETRIA GPS COM ESTRUTURAS DE DADOS

Relatório técnico do Projeto de Estrutura de Dados 2026

Paulo Sérgio Magri Júnior

Disciplina: Estrutura de Dados

Linguagem: C++17

Sorocaba

2026

PAULO SÉRGIO MAGRI JÚNIOR

SISTEMA DE TELEMETRIA GPS COM ESTRUTURAS DE DADOS

Relatório técnico apresentado à disciplina de Estrutura de Dados, do curso de Engenharia de Controle e Automação da Universidade Estadual Paulista “Júlio de Mesquita Filho”, Instituto de Ciência e Tecnologia, Câmpus de Sorocaba, como parte das atividades avaliativas do Projeto de Estrutura de Dados 2026.

Sorocaba
2026

RESUMO

Este relatório apresenta o desenvolvimento de um sistema simulado de telemetria GPS utilizando estruturas de dados em C++17. O sistema manipula registros compostos por timestamp, latitude, longitude, elevação e velocidade, simulando um fluxo contínuo de dados de localização e movimento. A implementação foi construída de forma incremental para atender aos requisitos do Projeto de Estrutura de Dados 2026, incluindo operações básicas de inserção, busca e remoção, operações adicionais de análise dos dados, benchmarks, testes sob restrições e comparação entre uma estrutura original e uma estrutura otimizada.

Foram implementadas cinco estruturas de dados: Lista Encadeada, Árvore AVL, Tabela Hash, Fenwick Tree e KD-Tree. As três primeiras foram utilizadas como estruturas tradicionais, enquanto Fenwick Tree e KD-Tree foram adotadas como estruturas fora da ementa. Os resultados experimentais foram registrados em arquivos CSV, permitindo análise de desempenho, memória, colisões e comportamento sob restrições. O sistema também possui menu interativo por terminal, facilitando a visualização e a arguição do projeto.

Palavras-chave: estruturas de dados; telemetria GPS; AVL; tabela hash; Fenwick Tree; KD-Tree; benchmark.

SUMÁRIO

- 1 Introdução
- 2 Objetivos
- 3 Metodologia
- 4 Dataset e tratamento dos dados
- 5 Estruturas de dados implementadas
- 6 Operações adicionais
- 7 Benchmarks e resultados
- 8 Testes com restrições
- 9 Estrutura otimizada
- 10 Visualização do sistema
- 11 Discussão geral
- 12 Conclusão
- Referências
- Apêndice A - Comandos de compilação e execução

1 INTRODUÇÃO

O desenvolvimento de sistemas computacionais modernos exige a organização eficiente de grandes volumes de dados. Aplicações de monitoramento, sistemas de transporte, robótica móvel, redes de sensores, plataformas industriais e sistemas embarcados frequentemente trabalham com dados gerados continuamente. Nesse contexto, estruturas de dados são fundamentais para permitir inserção, busca, remoção, atualização, indexação e análise com desempenho adequado.

O projeto desenvolvido neste trabalho simula um sistema de telemetria GPS. Cada registro representa uma leitura contendo informações temporais, espaciais e cinemáticas, possibilitando diferentes formas de organização e consulta. A escolha do domínio de telemetria foi motivada pela possibilidade de utilizar tanto estruturas clássicas, como Lista Encadeada, Árvore AVL e Tabela Hash, quanto estruturas mais especializadas, como Fenwick Tree e KD-Tree.

O foco do sistema não é produzir uma aplicação comercial, mas demonstrar domínio conceitual e prático sobre estruturas de dados. Por isso, a implementação foi acompanhada de análises de complexidade, benchmarks, testes sob restrições e justificativas metodológicas para evitar comparações inadequadas entre estruturas com finalidades distintas.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver um sistema simulado de telemetria GPS capaz de armazenar, organizar, buscar, remover, analisar e comparar registros utilizando diferentes estruturas de dados em C++¹⁷.

2.2 Objetivos específicos

- Implementar cinco estruturas de dados, sendo três tradicionais e duas fora da ementa.
- Realizar operações básicas de inserção, busca e remoção.
- Executar três operações adicionais sobre o dataset.
- Realizar benchmarks padronizados com diferentes tamanhos de entrada.

- Simular cinco restrições, contemplando memória, processamento, latência, dados e substituição algorítmica.
- Implementar uma estrutura otimizada e compará-la com sua versão original.
- Gerar arquivos CSV para análise dos resultados.
- Disponibilizar visualização por terminal para facilitar a apresentação e a arguição.

3 METODOLOGIA

A metodologia adotada seguiu uma abordagem incremental. Inicialmente foi definida a estrutura do registro GPS e a leitura do arquivo CSV. Em seguida foram implementadas as estruturas de dados, testando individualmente inserção, busca e remoção. Após a validação das estruturas, foram adicionadas operações adicionais, benchmarks, restrições, comparação de estrutura otimizada e menu interativo.

A organização textual deste relatório foi elaborada em formato técnico, com estrutura compatível com a apresentação de trabalhos acadêmicos em padrão utilizado pela UNESP e normalização baseada em orientações ABNT. A página da Biblioteca da UNESP Sorocaba disponibiliza guias de normalização, incluindo o Manual de Normalização de Trabalhos Acadêmicos da Rede de Bibliotecas Unesp e outros guias de apresentação tabular. A Rede de Bibliotecas da UNESP também mantém manual de normalização para trabalhos acadêmicos conforme normas ABNT.

3.1 Ambiente de desenvolvimento

O sistema foi desenvolvido em C++17 no ambiente GitHub Codespaces/Visual Studio Code, com compilação via g++. A execução principal foi realizada em ambiente Linux. O comando de compilação adotado foi:

```
g++ -std=c++17 -Wall -Wextra src/main.cpp -o bin/sistema
./bin/sistema
```

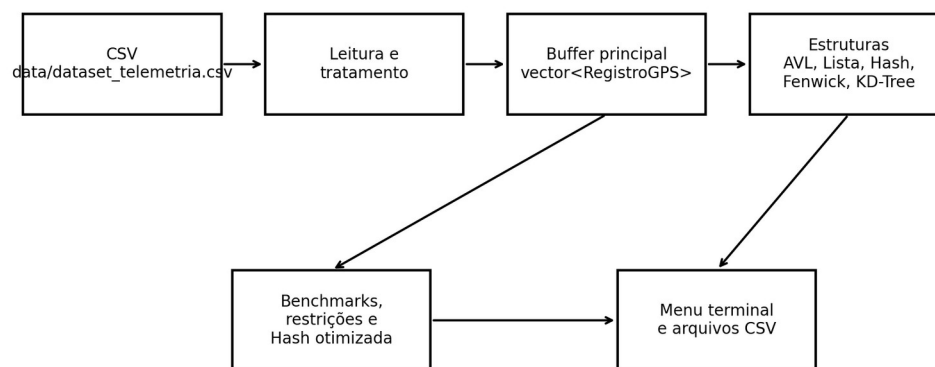
3.2 Organização do projeto

Após a implementação, os arquivos foram organizados por finalidade para facilitar a navegação no repositório e a avaliação do projeto.

Tabela 1 - Organização dos arquivos do projeto

Pasta/arquivo	Função no projeto
src/main.cpp	Código-fonte principal do sistema.
tools/gerador_dataset.cpp	Ferramenta auxiliar para geração de dataset sintético.
data/dataset_telemetria.csv	Dataset de entrada utilizado pelo sistema.
results/resultados_benchmark.csv	Arquivo com resultados dos benchmarks.
results/resultados_restricoes.csv	Arquivo com resultados dos testes de restrição.
results/comparacao_hash_otimizada.csv	Arquivo com comparação entre Hash original e Hash otimizada.
docs/relatorio.pdf	Relatório técnico final.
bin/sistema	Executável principal do sistema.

Figura 1 - Arquitetura geral do sistema



Fonte: elaboração do autor.

4 DATASET E TRATAMENTO DOS DADOS

O dataset utilizado contém registros de telemetria GPS em formato CSV. Cada linha representa uma leitura composta por timestamp, latitude, longitude, elevação e velocidade. O sistema exige um mínimo de 10.000 amostras válidas, e a execução final observada utilizou 15.000 registros carregados após o tratamento inicial.

Tabela 2 - Campos do registro GPS

Campo	Tipo no código	Descrição
timestamp	uint64_t	Instante associado ao registro.
latitude	double	Coordenada geográfica no eixo norte-sul.
longitude	double	Coordenada geográfica no eixo leste-oeste.
elevacao	float	Elevação associada ao ponto de coleta.
velocidade	float	Velocidade registrada no instante da leitura.

4.1 Validação e limpeza

Durante a leitura, são descartados registros com latitude fora do intervalo $[-90, 90]$, longitude fora do intervalo $[-180, 180]$, elevação fora da faixa física admitida, velocidade negativa ou valores inválidos. Após a validação, os registros são ordenados por timestamp, e timestamps duplicados são removidos. Essa etapa reduz inconsistências e evita que as estruturas sejam alimentadas com dados inválidos.

Tabela 3 - Resumo da execução observada do dataset

Métrica observada	Valor
Registros carregados	15.000
Timestamp inicial	1710000000000
Timestamp final	1710014999000
Velocidade média	19,3763
Velocidade mínima	0
Velocidade máxima	45

5 ESTRUTURAS DE DADOS IMPLEMENTADAS

Foram implementadas cinco estruturas de dados. Lista Encadeada, Árvore AVL e Tabela Hash representam estruturas tradicionais. Fenwick Tree e KD-Tree foram adotadas como estruturas fora da ementa, conforme a lista de opções do projeto.

Tabela 4 - Estruturas implementadas

Estrutura	Categoria	Finalidade no sistema
Lista Encadeada	Tradicional	Armazenamento sequencial e comparação com estruturas mais eficientes.
Árvore AVL	Tradicional	Indexação ordenada por timestamp.
Tabela Hash	Tradicional	Busca rápida por timestamp.
Fenwick Tree	Fora da ementa	Consulta de soma acumulada de ganho de elevação.
KD-Tree	Fora da ementa	Busca espacial por latitude e longitude.

5.1 Lista Encadeada

A Lista Encadeada foi utilizada como estrutura tradicional de armazenamento sequencial. Cada nó armazena o timestamp e o índice do registro correspondente no vetor principal. A inserção no final é eficiente quando há ponteiro para o último nó, mas a busca e a remoção por timestamp exigem varredura sequencial, com complexidade $O(n)$.

5.2 Árvore AVL

A Árvore AVL foi utilizada para indexar os registros por timestamp. Por ser uma árvore binária de busca autobalanceada, mantém altura proporcional a $O(\log n)$, garantindo inserção, busca e remoção logarítmicas. O sistema implementa rotações, atualização de altura, fator de balanceamento, destrutor e verificação de balanceamento.

5.3 Tabela Hash

A Tabela Hash original utiliza encadeamento separado para tratamento de colisões. A chave usada é o timestamp, e o valor armazenado é o índice do registro no vetor principal. A estrutura contabiliza colisões, pois esse é um indicador relevante para a eficiência da estratégia de hashing.

5.4 Fenwick Tree

A Fenwick Tree foi utilizada para somas acumuladas de ganho positivo de elevação. Essa estrutura permite atualizações e consultas de prefixo em $O(\log n)$, sendo apropriada para perguntas como o ganho de elevação em determinado intervalo da trajetória.

5.5 KD-Tree

A KD-Tree foi aplicada à busca espacial. Como os registros possuem latitude e longitude, a estrutura é adequada para localizar o ponto geográfico mais próximo de uma coordenada informada. A remoção foi implementada logicamente, marcando nós como inativos.

Tabela 5 - Complexidade teórica resumida

Estrutura	Inserção	Busca	Remoção	Observação
Lista Encadeada	$O(1)$	$O(n)$	$O(n)$	Busca sequencial.
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Balanceamento por rotações.
Hash	$O(1)$ médio	$O(1)$ médio	$O(1)$ médio	Depende da taxa de colisões.
Fenwick Tree	$O(\log n)$	$O(\log n)$ para soma	$O(\log n)$	Atualização para zero.
KD-Tree	$O(\log n)$ médio	$O(\log n)$ médio	Remoção lógica	Busca espacial.

6 OPERAÇÕES ADICIONAIS

Além das operações básicas de inserção, busca e remoção, foram implementadas três operações adicionais: estatísticas, detecção e filtragem de anomalias e classificação de movimento.

6.1 Estatísticas

O sistema calcula média, mínimo, máximo e desvio padrão para velocidade e elevação. Essas métricas servem para caracterizar o comportamento geral da trajetória e auxiliar na definição de limiares para anomalias.

6.2 Detecção e filtragem de anomalias

A detecção de anomalias considera velocidades acima de um limite calculado e saltos bruscos de elevação entre registros consecutivos. Essa operação simula uma etapa de tratamento de dados comum em sistemas de sensores, nos quais ruído e leituras inconsistentes podem comprometer a análise.

6.3 Classificação de movimento

A classificação de movimento agrupa registros em Parado, Lento, Moderado e Rápido. Essa classificação permite transformar valores numéricos em categorias interpretáveis para visualização e análise do fluxo de telemetria.

Tabela 6 - Critérios de classificação de movimento

Classe	Critério adotado
Parado	Velocidade menor que 1.
Lento	Velocidade entre 1 e 10.
Moderado	Velocidade entre 10 e 30.
Rápido	Velocidade maior ou igual a 30.

7 BENCHMARKS E RESULTADOS

Os benchmarks foram executados para medir tempo de inserção, busca, remoção, uso estimado de memória, colisões na Tabela Hash e escalabilidade. Os resultados quantitativos detalhados são gravados no arquivo results/resultados_benchmark.csv, gerado automaticamente durante a execução do sistema.

A metodologia de comparação respeita a finalidade de cada estrutura. Lista Encadeada, AVL e Tabela Hash são comparadas em operações de timestamp. Fenwick Tree é avaliada separadamente em consultas de soma acumulada, e KD-Tree é avaliada separadamente em busca espacial. Essa separação evita comparações inválidas entre estruturas que não resolvem o mesmo problema.

Tabela 7 - Agrupamento metodológico dos benchmarks

Grupo de análise	Estruturas	Operação avaliada
Busca por timestamp	Lista Encadeada, AVL, Hash	Inserção, busca e remoção por timestamp.
Soma acumulada	Fenwick Tree	Consulta de ganho de elevação em intervalo.
Busca espacial	KD-Tree	Ponto mais próximo por latitude e longitude.

7.1 Resultados observados

A execução final gerou o arquivo results/resultados_benchmark.csv. Como os tempos dependem do ambiente de execução, os valores numéricos devem ser analisados diretamente no CSV. Na execução observada, todos os módulos de benchmark foram concluídos com sucesso, e o sistema exibiu mensagem de conclusão da Parte 8, confirmando a criação do arquivo de resultados.

Tabela 8 - Síntese dos resultados de benchmark

Item verificado	Resultado observado
Arquivo de benchmark	Gerado em results/resultados_benchmark.csv
Comparação de timestamp	Lista, AVL e Hash separadas em grupo comparável.
Fenwick Tree	Avaliada em consulta de soma acumulada.
KD-Tree	Avaliada em busca espacial por latitude/longitude.
Observação metodológica	Comparações inválidas foram evitadas.

8 TESTES COM RESTRIÇÕES

Foram implementadas cinco restrições, contemplando memória, processamento, latência, dados e substituição algorítmica/estrutural. Os resultados foram gravados em results/resultados_restricoes.csv.

Tabela 9 - Restrições implementadas

Categoria	Restrição	Descrição
Memória	R5	Descarte automático de dados antigos.
Processamento	R7	Orçamento máximo de tempo por operação.
Latência	R13	Processamento em lote com atraso artificial.
Dados	R16	Perda de 20% das leituras.
Algorítmica/Estrutural	R21	Substituição de Hash por Lista Encadeada.

A restrição R5 avalia o comportamento do sistema quando é necessário descartar dados antigos para limitar uso de memória. A R7 mede operações sob limite de tempo. A R13 simula atraso por lote. A R16 remove uma fração dos dados, representando perda de leituras. A R21 evidencia a diferença entre uma estrutura eficiente, como Hash, e uma estrutura menos eficiente para busca, como Lista Encadeada.

Tabela 10 - Síntese dos testes com restrições

Resultado	Situação
Arquivo de restrições	Gerado em results/resultados_restricoes.csv
Categorias contempladas	Memória, processamento, latência, dados e algoritmo.
Restrição algorítmica	Comparação entre Hash e Lista Encadeada.
Validação final	Execução concluiu a Parte 9 com sucesso.

9 ESTRUTURA OTIMIZADA

A estrutura escolhida para otimização foi a Tabela Hash. A versão original utiliza encadeamento separado com nós alocados dinamicamente e ponteiros. A versão otimizada utiliza vetores contíguos, endereçamento aberto, sondagem linear e marcação de estados de posição, reduzindo o uso de ponteiros e melhorando a localidade de memória.

Tabela 11 - Comparação conceitual entre Hash original e otimizada

Característica	Hash original	Hash otimizada
Estratégia de colisão	Encadeamento separado	Endereçamento aberto com sondagem linear
Armazenamento	Nós dinâmicos com ponteiros	Vetores contíguos
Localidade de memória	Menor	Maior
Remoção	Remoção de nó na lista	Marcação de posição removida
Arquivo de resultados	comparação no mesmo CSV	results/comparacao_hash_otimizada.csv

A comparação experimental avalia inserção, busca, remoção, memória estimada e colisões. A execução observada concluiu a Parte 10 com sucesso e gerou o arquivo results/comparacao_hash_otimizada.csv. A análise final deve considerar que a otimização pode melhorar localidade de memória, mas a sondagem linear também pode ser sensível ao fator de carga.

10 VISUALIZAÇÃO DO SISTEMA

O sistema possui menu interativo por terminal, permitindo explorar o status do sistema, visualizar registros, buscar por timestamp usando Hash, buscar ponto geográfico próximo usando KD-Tree, consultar ganho de elevação usando Fenwick Tree, exibir operações adicionais e listar arquivos CSV gerados.

Tabela 12 - Opções do menu interativo

Opção	Função
1	Exibir status do sistema.
2	Exibir primeiros registros.
3	Buscar registro por timestamp usando Hash.
4	Buscar ponto geográfico mais próximo usando KD-Tree.
5	Consultar ganho de elevação usando Fenwick Tree.
6	Exibir operações adicionais.

7	Exibir arquivos CSV gerados.
0	Sair.

Durante os testes, foi necessário ajustar a função de leitura do menu para utilizar getline, melhorando a interação no terminal. Após a correção, a visualização interativa funcionou corretamente.

11 DISCUSSÃO GERAL

Os resultados obtidos mostram que a escolha da estrutura de dados depende diretamente da operação desejada. A Lista Encadeada possui implementação simples e inserção eficiente no final, mas apresenta desvantagem em buscas por timestamp. A AVL oferece desempenho logarítmico e previsível, sendo apropriada para cenários em que a ordenação das chaves é importante. A Tabela Hash apresenta busca média eficiente, mas depende da função de dispersão, da taxa de colisões e do fator de carga.

A Fenwick Tree não substitui uma estrutura de busca geral. Sua utilidade está em consultas acumuladas, como soma de ganhos de elevação. A KD-Tree também não deve ser comparada diretamente com Hash por timestamp, pois sua finalidade principal é busca espacial em dados multidimensionais. Essa distinção é essencial para manter a validade metodológica dos benchmarks.

A implementação da Hash otimizada reforça a importância da organização interna da memória. Ao substituir nós alocados dinamicamente por vetores contíguos, busca-se melhorar localidade de memória e reduzir overhead de ponteiros. Entretanto, a eficácia da otimização depende do fator de carga e da distribuição das chaves.

12 CONCLUSÃO

O projeto implementou um sistema de telemetria GPS em C++17 com cinco estruturas de dados, operações básicas, operações adicionais, benchmarks, restrições, estrutura otimizada e visualização por terminal. A implementação cumpriu os principais requisitos técnicos do Projeto de Estrutura de Dados 2026.

As estruturas tradicionais - Lista Encadeada, AVL e Tabela Hash - permitiram comparar diferentes estratégias para busca por timestamp. As estruturas fora da ementa - Fenwick Tree e

KD-Tree - ampliaram o sistema para consultas acumuladas e buscas espaciais. A Hash otimizada possibilitou discutir localidade de memória, tratamento de colisões e trade-offs entre tempo e memória.

Conclui-se que não existe uma estrutura universalmente superior. A adequação depende do tipo de consulta, do padrão de atualização, das restrições do ambiente e da finalidade do sistema. O projeto evidenciou essa relação por meio de implementação, testes e análise comparativa.

REFERÊNCIAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. Estruturas de dados e algoritmos em C++. 2. ed. Porto Alegre: Bookman, 2013.

SEDGEWICK, Robert; WAYNE, Kevin. Algorithms. 4. ed. Boston: Addison-Wesley, 2011.

KNUTH, Donald E. The Art of Computer Programming: Sorting and Searching. 2. ed. Boston: Addison-Wesley, 1998.

BENTLEY, Jon Louis. Multidimensional binary search trees used for associative searching. Communications of the ACM, v. 18, n. 9, p. 509-517, 1975.

UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA FILHO”. Coordenadoria Geral de Bibliotecas. Rede de Bibliotecas. Manual de normalização de trabalhos acadêmicos: apresentação: ABNT. São Paulo: UNESP, 2023.

UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA FILHO”. Biblioteca - Câmpus de Sorocaba. Guias de normalização. Disponível em: <https://www.sorocaba.unesp.br/>. Acesso em: 1 jul. 2026.

UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA FILHO”. Projeto de Estrutura de Dados 2026. Proposta da disciplina fornecida pelo docente, 2026.

APÊNDICE A - COMANDOS DE COMPILAÇÃO E EXECUÇÃO

Compilação do sistema principal:

```
g++ -std=c++17 -Wall -Wextra src/main.cpp -o bin/sistema  
./bin/sistema  
ls results
```

Arquivos esperados na pasta results: resultados_benchmark.csv, resultados_restricoes.csv e comparacao_hash_otimizada.csv.